

CI/CD Report

Student: 23127396 · **Pipeline:** `.github/workflows/api-tests.yml` · **Runner:** `ubuntu-latest`

Pipeline configuration

Triggers on every push and PR to `main`, plus manual dispatch. Steps, in order:

1. Checkout, Node 22 setup.
2. `npm ci` (test tooling — Newman) and `npm --prefix sut/backend ci` (the SUT's own deps).
3. **Start the SUT**, then `node scripts/wait-for-sut.js` — the SUT's own `database.js` DROPs and re-seeds every table at module load, so the server must be listening *before* anything seeds fixtures on top of it (start → wait → seed, never seed → start).
4. **Seed deterministic fixtures** (`scripts/seed-api-data.js`) — 3 users with fixed roles/profile fields, 10 orders (one per FR-10 state, per owner), 6 coupons (the SUT's 4 defaults plus `INACTIVE10` / `EXPIRETODAY`, needed by API 2's `A2-S2-03` and `A2-EX-06`), `coupon_usage` cleared.
5. **Harness self-check** (`npm run test:smoke`) — confirms the `X-Student-Id` header and the environment resolve before spending time on the graded collections.
6. **Detect graded collections**, then **run them** via `node scripts/run-newman.js`, which re-seeds *between* every collection (see the quarantine mechanism below for why the run can still be green) and uploads every `reports/*.html` / `.json` as a build artifact regardless of outcome.
7. Job summary (see below) and SUT teardown, both `if: always()`.

The quarantine mechanism (why a "failing" Newman run can still be a green build)

Almost every collection in this suite deliberately asserts **spec-correct** behaviour against a **confirmed SUT defect** — that is the whole point of the generate → audit → extend → execute pipeline: a case that got silently rewritten to match the bug would delete the finding. That means Newman's own exit code is nonzero on essentially every normal run of this suite, and can't drive the CI gate by itself.

`scripts/run-newman.js` resolves this without ever touching a test case's assertion:

1. Run the collection. If Newman exits 0, it passes outright.
2. If Newman exits nonzero, parse the JSON report it just wrote and extract the case ID (`A2-S4-03`, `A1-EX-06`, ...) of every failing assertion.
3. Compare that set against [postman/known-defects.json](#) — a manifest mapping each collection to the exact case IDs it is *allowed* to fail, each tagged with the bug it targets (cross-referenced to `docs/api*/audit.md` / `extended.md`).
4. The collection passes the gate **iff every failing case ID is in that manifest**. Any failure that *isn't* listed still fails the build — a real regression is still red.
5. The manifest is derived from actually-observed Newman JSON reports, not hand-guessed; if a listed defect stops reproducing (say, someone fixes `BUG-09`), the run prints a `NOTE:` line naming it rather than silently going quiet, so the manifest doesn't quietly drift from reality.

This is the "known-defect folder quarantined + documented" requirement (HW06 §6, `PLAN.md` step C1) — "folder" here is a manifest file rather than a physically separate collection, because the EXPECTED-FAIL cases are woven throughout each collection's S2-S5/EX folders by design (grouped with the dimension they test, not segregated by outcome), and physically relocating ~55 already-verified, carefully-fixture-sequenced items across six collections would have been strictly riskier than adding one small, auditable comparison step.

The GitHub Actions job summary makes this visible without opening the manifest: it prints a per-collection request/assertion/failed-count table and an explanatory note pointing at `postman/known-defects.json` , so a nonzero **Failed** column next to a green checkmark reads as "documented," not as a hidden failure.

Sample runs

Run	Commit	Result	Link
All API test cases passing (known defects quarantined)	a6c3e0f "ci: full API suite passing"	 green	Run 32123101092 (34s)
One test case deliberately failing (unquarantined)	ff425eb "test: deliberate failing case for CI evidence (HW06 §6)"	 red	Run 32123317185 (31s)
Revert — restores the green baseline	2fc5e7d "revert: restore correct expectation"	 green	Run 32123424684 (33s)

Screenshots: [evidence/ci-c1-green-run-header.jpg](#) + [evidence/ci-c1-green-run-summary.jpg](#) for the green run; [evidence/ci-c2-red-run-header.jpg](#) + [evidence/ci-c2-red-run-summary.jpg](#) for the red run. Each pair captures the run header (status, duration) and the job-summary table underneath it — note `ap11-usersme` 's **Failed** column goes from 19 (green) to 20 (red), the one deliberately broken assertion.

The deliberate failure (`ff425eb`) flipped exactly one assertion — `A1-S2-01` 's expected status from 200 to 201, on a request the SUT genuinely (and correctly) answers with 200 — so the mismatch is a one-line, reviewable diff, and it is **not** listed in `postman/known-defects.json` . The CI run correctly went red on it (`Run graded collections` step failed, job exit code 1), confirming the quarantine mechanism only shields *documented* defects and still fails the build on a real regression. The revert commit is a byte-for-byte inverse of the flip.